

AEGIS

Problem, Solution & Three Scan Modes

1. The Problem

Application security is usually split across many separate tools. One tool tests the live website, another scans the source code, and another checks cryptography. These tools often produce separate lists of alerts without understanding how the findings are connected.

This creates three major problems:

- **Runtime blind spot:** A live scanner can show that an application is vulnerable, but it normally cannot explain which source-code component caused the problem.
- **Source-code blind spot:** A code scanner can find a dangerous pattern, but it cannot always prove that the deployed application is actually exploitable.
- **Cryptography is disconnected:** Weak algorithms, bad key handling, certificates, and insecure crypto settings are often reported separately from application-security findings.

The result is a large collection of findings instead of one clear answer to:

“What is actually wrong with my application, where is the root cause, and what should I fix first?”

2. The AEGIS Solution

AEGIS is an adaptive application-security platform. It accepts two possible inputs: a **live application URL** and a **source-code repository**. Depending on what the user provides, AEGIS automatically runs the appropriate security engines.

AEGIS has three security components:

- **Agent 1 — Runtime Security Agent:** tests the live website/API from the outside for authentication problems, BOLA/broken access control, SSRF, basic injection, and security misconfiguration.
- **Agent 2 — Code Security Agent:** scans the repository to understand source code, dependencies, secrets, configuration, and possible security weaknesses.
- **Crypto Engine:** scans the repository for cryptographic assets and weaknesses such as weak algorithms, insecure modes, bad key handling, certificates, and quantum-vulnerable cryptography.

The important difference

AEGIS does not simply show three independent scanner outputs. It combines their evidence. For example, Agent 1 may prove that User A can access User B's data, while Agent 2 identifies the exact function where authorization is missing. AEGIS can present both as one stronger finding.

Simple idea: Agent 1 tells us *what is happening*. Agent 2 tells us *why it is happening*. The Crypto Engine tells us *what is wrong with the cryptography*. AEGIS connects all of this information.

3. Three Scan Modes

The user does not manually choose agents. AEGIS checks which inputs were provided and automatically selects the correct mode.

MODE 1 — URL ONLY

User provides: Live application URL **Runs:** Agent 1 only

How it works: AEGIS connects to the deployed application, discovers endpoints, tests authentication and authorization, performs controlled SSRF/injection checks, checks security configuration, collects evidence, and generates a runtime-security report.

Limitation: No repository means no source-code root-cause analysis and no repository-level cryptography analysis.

MODE 2 — REPOSITORY ONLY

User provides: GitHub/repository URL **Runs:** Agent 2 + Crypto Engine

How it works: Agent 2 examines source code, dependencies, secrets, APIs, authentication/authorization logic, and configuration. At the same time, the Crypto Engine searches for cryptographic algorithms, keys, certificates, crypto libraries, insecure modes, and other crypto risks.

Result: Source-level security findings plus a cryptography assessment. Runtime exploitation is not claimed because no deployed URL was supplied.

MODE 3 — URL + REPOSITORY

User provides: Both deployed URL and repository **Runs:** Agent 1 + Agent 2 + Crypto Engine

How it works: Agent 1 tests the live application. Agent 2 analyzes the source code. The Crypto Engine analyzes cryptography. AEGIS then combines the evidence and connects the live application, source code, vulnerabilities, and crypto assets.

Example: Agent 1 proves BOLA on GET /api/users/{id}. Agent 2 finds that the corresponding source function retrieves the user object without an authorization check. AEGIS connects these results and reports the vulnerability together with its source-code root cause.

This is the full AEGIS assessment.

User Input	Agent 1	Agent 2	Crypto	Result
URL only	✓	—	—	Runtime report
Repository only	—	✓	✓	Code + Crypto report
URL + Repository	✓	✓	✓	Full correlated report

4. AEGIS in One Picture

URL → Agent 1 → Runtime Evidence

Repository → Agent 2 → Source Evidence

Repository → Crypto Engine → Crypto Evidence

All available evidence → Correlation → Risk → Remediation → Final Report

The architecture is modular: with only a URL, AEGIS still provides runtime security value; with only a repository, it provides source and cryptographic analysis; with both, it provides the deepest assessment by connecting runtime behavior with source-level causes.